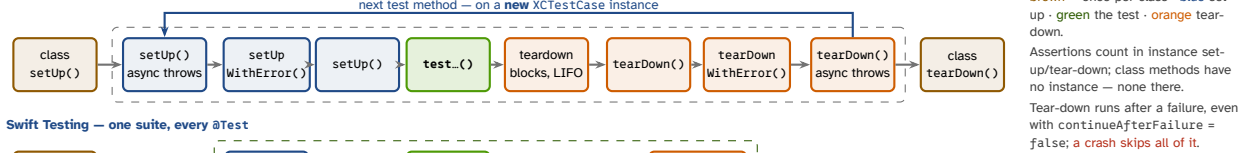
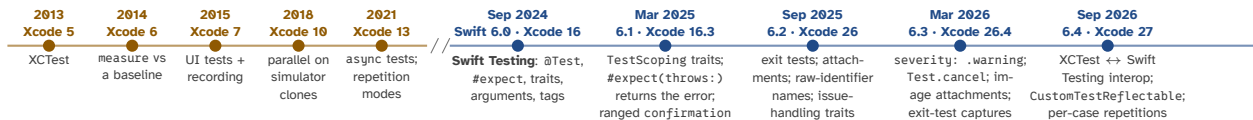
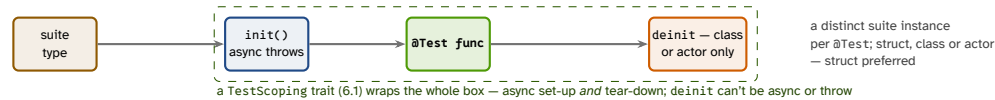


In one line: XCTest: subclass XCTestCase, name methods test..., assert with XCTAssert*; a fresh instance per test method. **Swift Testing** (open source, in the Swift toolchain, all Swift platforms): @Test functions, any type is a suite, two macros #expect / #require, traits and arguments — run **in parallel, in one process** by default. Both live in one target; UI automation, XCTestMetric performance tests and Objective-C stay in XCTest.

XCTest — one class, every test method



Swift Testing — one suite, every @Test



XCTest — the rules

- Test method:** instance method of an XCTestCase subclass, no parameters, no return value, name starts with lowercase test; may be throws or async. Synchronous tests run on the **main actor**; async set-up/tear-down and blocks run on an arbitrary actor unless marked @MainActor.
- Set-up:** setUpWithError() when it can throw — a throw is recorded as a failure, no try!; setUp() async throws for async work. addTearDownBlock { } puts clean-up next to the code that needs it.
- continueAfterFailure (default true); false ends the method at the first failure — the other tests still run. try XCTUnwrap(x) fails and stops one test; x! on nil crashes the process: no tear-down.
- Skip:** throw XCTSkip("...") · try XCTSkipIf(c) / Unless(c). **Known failure:** XCTExpectFailure("...") — isStrict defaults to true, so a failure that stops happening becomes an issue. Xcode 26: an XCTIssue of warning severity does not fail the test.
- executionTimeAllowance: default 600s, rounded up to whole minutes, active only when timeouts are on (test plan or -test-timeouts-enabled YES).
- XCTAttachment + add(:): Lifetime defaults to .deleteOnSuccess — kept only for failures unless .keepAlways.
- Parallel:** the simulator is cloned; each runner process takes *whole classes*, one test at a time — one giant class never parallelises.

Performance tests — XCTest only

measure { } times the block; measure(metrics:options:) records any of XCTClockMetric (time) · XCTCPUMetric · XCTMemoryMetric (physical) · XCTStorageMetric (logical writes) · XCTOSSignpostMetric (a signposted interval) · XCTApplicationLaunchMetric · XCHitchMetric. The block runs iterationCount + 1 times; the first is ignored (warm caches). A **baseline** = average + allowed standard deviation, set from the report and stored **per device configuration**.

The leak test — weak, scope, nil

```
func test_viewModel_isReleased() {
    weak var weakSUT: FeedViewModel?
    do {
        let sut = FeedViewModel(api: StubAPI())
        weakSUT = sut // observe, don't retain
        sut.start()    // may build a cycle
    }                 // last strong ref gone
    XCTAssertNil(weakSUT) // Swift Testing: #expect(weakSUT == nil)
}
func trackForLeaks(_ o: AnyObject, line: UInt = #line) {
    addTearDownBlock { [weak o] in XCTAssertNil(o, line: line) }
} // after the test body
```

Mixing the two (Swift 6.4, ST-0021)

SWIFT_TESTING_XCTEST_INTEROP_MODE: none · limited (an XCTAssert failing in a @Test = warning) · complete (= failure) · strict (fatalError). Toolchain < 6.4: none — the failure was silently lost; packages with tools-version ≥ 6.4 get complete, older ones limited; Xcode 27 sets it per test plan.

Traps

- ! trackForLeaks on a SUT kept in a var sut property always fails: teardown blocks run *before* tearDown() nils it — build it in a local makeSUT().
- ! Shared statics, singletons, UserDefaults · standard race once tests run in parallel — inject them, or .serialized as the last resort.
- ! .timeLimit / executionTimeAllowance catch hangs, not slowness — that is measure's job.

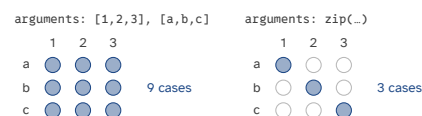
XCTest → Swift Testing

XCTest	Swift Testing
XCTAssertTrue(x)	#expect(x)
XCTAssertEqual(a, b), ...Nil, _Identical, ...GreaterThan	#expect(a == b), x == nil, ==, > — one macro; a failure shows every operand's value
XCTAssertThrowsError	let e = #expect(throws: E.self) { try f() } — e: E? (6.1); (any Error).self for any
XCTAssertNoThrow	#expect(throws: Never.self) { try f() }
try XCTUnwrap(x)	try #require(x) — throws ExpectationFailedError, ends this test
XCTFail("...")	Issue.record("..."); severity: .warning (6.3)
XCTSkipIf	.enabled(if:) · .disabled("why"); mid-test try Test.cancel() (6.3)
XCTExpectFailure	withKnownIssue { } — no issue ⇒ records one, unless isIntermittent: true
XCTestExpectation	await confirmation(expectedCount: 1...) { c in } — counted when the closure returns; it never waits
setUp / tearDown	init() / deinit; shared async — TestScoping trait
executionTimeAllowance	.timeLimit(.minutes(n)) — minutes only, shortest wins
XCTAttachment	Attachment.record(value, named:) (6.2)
measure, XCUIApplication, Objective-C	— stay in XCTest

Swift Testing — the rules

- @Test on a function at file scope or in a type; async, throws, @MainActor allowed. Name: @Test("...") or a raw identifier func `adds two numbers`() (6.2).
- A type holding tests is a suite; @Suite only adds a name or traits. Zero-argument init required; never @available on a suite. Suite traits (tags, .disabled) are inherited.
- #expect expands to code capturing the source text and each subexpression's value; #require also throws. Mid-test failure → Issue.record.
- Parallel, randomised, in-process** (tasks), on devices too. Tests no longer default to the main actor. .serialized — inherited by nested suites; on a parameterized test its cases run one by one.
- Run:** swift test runs both frameworks; --filter / --skip take a regex; --no-parallel 6.4: --maximum-repetitions n --repeat-until pass|fail repeats only the matching cases — as Xcode 27's repetition mode now does.
- Traits:** .enabled(if:) · .disabled · .bug("url") · .tags(.slow) with extension Tag { @Tag static var slow: Self { (test plans include/exclude by tag) · .timeLimit · .serialized · .compactMapIssues / .filterIssues (6.2) · a custom TestScoping (6.1).
- Exit tests** (6.2): await #expect(processExitsWith: .failure) { _ } — the closure is main() of a child process of the same test binary; .success · .failure · .exitCode(n) · .signal(n); may observe stdout/stderr. Captures (6.3): listed, Sendable + Codable. macOS, Linux, FreeBSD, OpenBSD, Windows — **not iOS**.
- Attachments:** String, bytes, Data, Encodable (6.2); CGImage, UIImage, NSImage, CIImage as .png/.jpeg (6.3); Transferable (6.4). Xcode: test report + .xcresult; swift test --attachments-path.

Arguments — one test case each



Each element is its own case, run in parallel, shown and re-runnable alone (if the argument is CustomTestArgumentEncodable, RawRepresentable, Encodable or Identifiable). At most **two** collections — every combination; zip pairs them.